

Top Insights from the Chat

Questions from the cohort, grouped by theme and attributed by name

Session 7 produced one of the busiest chats of the cohort so far, and a large share of it was substantive. What follows is the signal: the questions that would help anyone in the cohort, grouped by theme and attributed to the person who asked. Answers come from Srikanth, Dhaval, Krishna and the Codebasics team where one was given in session, and from fellow participants where they got there first, which happened a lot today.

1. Choosing Between LangChain and LangGraph

This was the single most asked theme of the session, and it came from all directions before, during and after the hands on portion.

Asked by	Question	Answer or takeaway
Pujan	Can we say LangChain is good for RAG based applications and LangGraph is better for agents?	Close but not quite the right cut. LangChain is better for generic LLM applications. When you have a specific workflow, LangGraph is preferred.
Gourav	When do we use LangChain and when LangGraph?	Same answer, and it held all session: generic LLM application means LangChain, defined workflow means LangGraph.
Ankit Garg	Cannot we use LangChain for complex workflows?	You can, but it gets awkward. Human in the loop is possible in LangChain and difficult, because the flow stays probabilistic.
Ankit Garg	What can only be achieved with LangGraph and not LangChain?	The sharpest formulation of the day: when you know the flow, LangGraph wins. When you do not know the exact flow, LangGraph gets complex and LangChain is better. Syed's reply put it as a test: can you build this same thing in LangChain?
Ankit Garg	As a new learner it is hard to know upfront. Any guidance?	Judgement comes from building. Dhaval's rule of thumb: if the agent is critical and complex, prefer LangGraph, because you do not want a simple

Asked by	Question	Answer or takeaway
		agent hallucinating on something where the cost of a mistake is high.
Sandip	LangChain gives create_agent as an abstraction layer. Does LangGraph give more parameters or methods to control agent behaviour?	Yes, exactly. Krishna added the precise version: create_agent is an abstraction, LangGraph gives finer control over single or multiple agents, their workflow and their state.
amit s / Ankit Gupta	Which one do companies prefer, and which is more widely used in industry?	No specific numbers were offered, but both are used widely. It depends on the use case.
Sharath Sheshadri	We can build this kind of workflow in n8n. How do we decide between n8n and LangGraph?	The graph is genuinely the same shape. n8n is low code and no code, so you are bounded by the integrations it ships. LangGraph is a coding framework, so you get tighter control and full customisation.
chaithanya krishna	People say each LangChain update breaks things and some are moving off it. Does the same apply to LangGraph?	Not answered live. Worth raising again, and Murthy's related observation that langchain community is deprecated is a real signal that notebook dependencies need periodic refreshing.

Key Insight (from Saurabh)

Written in the chat before the hands on even started, and it holds up against everything taught afterwards: LangChain builds simple LLM applications and processes such as document processing, RAG, prompting and tool calling. LangGraph manages and connects multiple LangChain processes, letting the application decide which process runs next based on the query, the previous steps and the results.

2. Tools, Docstrings and How an Agent Chooses

Asked by	Question	Answer or takeaway
Sharath S	Can the LLM also read the code and get the functionality?	In theory yes, these models are strong at code. In practice you would burn tokens re reading the implementation on every decision. The docstring is the cheap summary.
Sandip	Then the docstring must be meaningful?	

Asked by	Question	Answer or takeaway
		Srikanth: yes, it is basically a prompt. Sandip's own follow up is the memorable version: reading a docstring is faster than reading code.
Gourav	Who writes the content in the docstring?	We do, the developers creating the tools.
Yasmin Iqbal	Is it not necessary to state the input and the return response in the docstring?	It is suggested, because it tells the LLM how to use the tool and what it gets back. Strictly, the function signature already carries the argument names and return type.
Ravindra Devagiri	If the LLM has more than one tool, how does it decide? Are tags needed?	No tags. It reads all the tool descriptions and picks the one that best fits the task to be done.
Sanjeev Rohila	So annotations also help write efficient agents?	Yes. Type annotations and the signature feed the same decision the docstring guides.
supreet kaur	Why is the weather tool returning a hardcoded "it is always sunny" instead of real weather?	It is a teaching stub. The point of the example is the routing decision, not the weather API.

3. State and Reducers

Asked by	Question	Answer or takeaway
Amey Barbadikar	Is state like a Pydantic class?	Krishna: yes, you can think of LangGraph state as a Pydantic class that stores the shared data passed between nodes.
Yasmin Iqbal	Is SimpleState a model created from this notebook or a fixed model?	Neither. SimpleState is a user defined state schema written with TypedDict that specifies the data shared between nodes. It is not a model.
Amit Kumar	Does the structure of SimpleState stay the same across all solutions, or is it customised?	Always customised to the requirement. Treat the five fields in this notebook as the research agent's fields, nothing more.
Ashish	Can you repeat the difference between add_messages and operator.add?	Append versus concatenate. One new value onto an existing list is add_messages. Joining a whole new list onto the existing list is operator.add. Vivek's shorthand: it is like list.extend.

Asked by	Question	Answer or takeaway
Subhadeep Majumder	Can add_messages carry guardrails that block a message from entering the list?	Not currently, but you can create a custom node that validates or rejects the message before it ever reaches add_messages.
Shiba	If we are adding or concatenating, why is it called a reducer? It sounds like we are deducting something.	Because two values go in, old and new, and one comes out. Multiple values reduce to one final state value.
Pawan	Please explain how the reducer is working here.	A reducer is just a function attached to a state field through the annotation. Every incoming value for that field is routed through it before the state is updated.

4. Nodes and What They Return

This was the densest cluster of the session, and most of it came down to one idea: a node returns only what it changed.

Asked by	Question	Answer or takeaway
Sharath S	Should we not return <code>{state['draft']: response.content}</code> instead of <code>{"draft": response.content}</code> ?	No. <code>state['draft']</code> is the current value, not the field name. A node returns a partial state update, so the keys must be the state field names.
Renganathan M	In node two we are not updating the dictionary, it just returns the score. Is every node not supposed to update the state?	Every node does update the state, but only the fields it returns. LangGraph merges the partial update into the existing state automatically.
Harshit	The nodes are not updating the state. How will it be updated?	Same answer. The merge is done by the framework, not by the node.
Himesh Sarkari	When we return from <code>generate_draft</code> , will the draft actually be updated with the value?	Yes.
abhiram	Are these prompts user prompts or system prompts?	A mix. The instruction text ("write a short paragraph about") is the system prompt, and the topic pulled from the state is the user input.
Venkateswara Rao K	Why did we not increment the iteration when scoring the draft?	Iteration does not count node executions. It counts drafts. <code>generate_draft</code> sets it to 1,

Asked by	Question	Answer or takeaway
		revise_draft increments it afterwards, and scoring does not produce a new draft.
abhiram	Does iteration mean a version number of the draft?	Effectively yes, on the same logic as above.
Abdoul	Does node two, the scorer, know the topic, or did it infer from response.content?	Chaithanya answered it well: it reads the shared state. The first node returned the draft into the state, and the score node reads state["draft"] from there.
Ashu Swami	Do we not have to pass input arguments when adding nodes, or is that just for tracking?	No arguments are passed at add_node time. You register a name and the function. LangGraph passes the state in when it runs.
Aditya Bharadwaj	Is a node similar to @tool in LangChain?	Structurally yes, both are plain Python functions. The difference is the role: a tool is something an agent may choose to call, a node is a step the graph executes.
Abhisekh	Are generate, score and revise standard nodes for all LangGraph workflows?	No, only for this example. You name and design your own nodes per use case.
RAVI	Must we always keep the same node count or flow? Can nodes have different names?	Any names, any count. There is no limit and no fixed shape.

5. Edges, Conditional Edges and Routers

Asked by	Question	Answer or takeaway
Ashu Swami / imtiaz	Why revise to score, and not the other way?	Because after a revision the new draft has to be scored again. Amit Patil answered it in one line in the chat: to score after revision. The direction of add_edge is what creates the loop.
Saurabh	Can we connect multiple nodes in a single add_node, or only two at a time?	Not in one call, but you can add several edges out of the same node. That is exactly how you fan out to parallel work, for example searching Google, Wikipedia and news at once.

Asked by	Question	Answer or takeaway
Renganathan M	Can nodes run in parallel once one node finishes?	Yes, if the nodes are independent. The three way search example is the canonical case: those do not need to be sequential.
Akhilesh	Is the sequence of adding conditional edges what defines where the condition applies?	The condition applies at the node you name as the source of the conditional edge, and the mapping decides the destinations.
imtiaz	Why are we adding conditional edges through the router function? Can we not define them after the edges?	<code>add_edge</code> blindly connects two nodes with no validation. <code>add_conditional_edges</code> has to run logic to build the edge, so the router and the routing map are passed into that method as parameters.
Amey Barbadikar	So the router has read only access to state and cannot modify it?	Yes, exactly. Node functions return dictionaries and write to the state. Router functions return a string and never do.
Raghavendra	What score sends it to the approved state?	Whatever threshold you set in the router. In the notebook it is a score above 7.
Pujan	Where do we mention that <code>needs_work</code> goes to revise?	In the mapping dictionary passed to <code>add_conditional_edges</code> . The string the router returns is the key, the node name is the value.
saikrishna	If node one generates something and node two scores it 2 out of 10, revise sends it back to node two. Will node two ever send it back to node one?	No. In this graph, <code>generate</code> runs once. The correction loop is score to revise to score, and it never returns to generate.
bergelin kamdem	Can we say there are two kinds of node, ones that modify state and ones that evaluate state to decide whether to continue?	That is a clean way to hold it, as long as you remember that the second kind is technically a router function, not a node registered on the graph.

6. The approved Flag and `give_up`: The Best Debate of the Session

Several people spotted the same thing independently, chased it, and got the design point out of it. Worth reading even if you did not notice it live.

Asked by	Question	Answer or takeaway
Syed Mizbahuddin	We never stored whether we exited because it was approved or because we gave up. Also approved: <code>bool</code> is never set to <code>True</code> , and <code>print(result['approved'])</code> confirms it.	Correct on both counts, and the reason is structural: the router returns a string and cannot write to state, so nothing in this graph ever sets it.
Abhisekh	The approved attribute was never updated in the state. What is the use of it then?	It is initial state, not output. If you want the exit reason recorded, a node has to write it, not the router.
Aditi Reddy	What is the significance of approved: <code>False</code> in the invoke statement if it never gets updated?	Mugdha answered it directly: it is the initial state.
imtiaz	Why do we not have a <code>give_up</code> node explicitly in the workflow diagram?	The if else comparison: you do not always draw both branches. Give up is an internal decision that ends execution, so it needs no node of its own.
Raghavendra	Confused between approved and <code>give_up</code> , since both go to the end state.	Same destination, different reason. Approved means the score cleared the bar. Give up means the iteration cap was hit, so stop regardless.
supreet kaur	Why did we not use <code>create_agent</code> here?	Because this approach gives you lower level control over the agent's behaviour. Syed added the framing that makes it click: assume this is the internals of <code>create_agent</code> , the ReAct loop.

Key Insight (from Syed Mizbahuddin)

Syed's point deserves keeping as a design habit, not just an answer. If you want to know later *why* a run ended, approved or exhausted, you have to write that fact into the state from inside a node. The router knows the reason and throws it away. That is a real gap in any production version of this graph.

7. Memory, Checkpointers and Human in the Loop

Asked by	Question	Answer or takeaway
Aditi Reddy	Why do you need memory when you are already updating state and carrying it across nodes?	State lives for one run, from start node to end node, and then it is gone. Memory is what

Asked by	Question	Answer or takeaway
		survives so a later run knows what happened and where it left off.
Ankit Gupta	Please revise memory and human in the loop, I am confused.	Srikanth's chat answer is the cleanest summary given: memory is what the system remembers, like telling ChatGPT your name. HITL is the system waiting for you, like Gmail creating a draft and not sending it until you say so.
Yasmin Iqbal	Can we control what the memory stores and what it does not?	Yes, and you should. The problem to avoid has a name: memory poisoning. Remember only the key and necessary information.
Sandip	But if we want proper context, must we not avoid skipping things from memory?	It is a trade. Too little context and the agent forgets. Too much and stale facts keep influencing answers alongside recent ones. Deliberate selection is the job.
Raju S	Is draft_email also saved in MemorySaver? How, with what keys and indexing strategy?	Everything in the state is saved, because the checkpointer needs the full picture to resume. Once you attach an external database instead of MemorySaver, you get control over what is persisted.
Sandip	With config, are we asking it to maintain memory?	The config carries the thread ID, which identifies which running graph you mean. The checkpointer is what stores. They work together.
Pujan	When it interrupts, should it not ask input from the user? Here we are sending an edited message.	That is the same thing, seen from the code side. In a UI the interrupt renders the buttons and the text area. In the notebook we simulate the click by resuming with a Command carrying the value.
Akhilesh	Can you repeat how the command would be issued?	Invoke the graph again with the same config and thread ID, passing a Command with the human's response. The graph resumes from the checkpoint rather than restarting.
Sharath Sheshadri / Pawan	Why AIMessage here and not BaseMessage, and when does the system create one versus the developer?	In production the confirmation genuinely comes back from the LLM, so it is an AI message. Here the send is simulated, so we inject an AIMessage by hand to make the history look the way it will look in production. The framework, not the model, decides who said what.

Asked by	Question	Answer or takeaway
Abhisekh	How do we handle the UI interruption options, the edit button handler and discard button handler, from the LangGraph nodes?	Expose each command as an API endpoint. When a button is pressed, that endpoint invokes the graph with the same config and thread ID and the matching Command. Srikanth also pointed at LangChain's Agent Chat UI in the LangGraph docs if you want a ready built front end.

8. Subgraphs, Reuse and Parallelism

Asked by	Question	Answer or takeaway
Navneet Singh	Can we create each node as a separate service so it can be deployed and reused by different graphs?	Yes. That is the subgraph pattern, and it was built live right after this question.
chaithanya krishna	Can we think of it as calling another function in Python, or a CTE in SQL, reusing an existing workflow as a child inside a parent?	That is a fair mental model. The parent node's body invokes the compiled child graph.
Mugdha	Can we pass parent graph state to the subgraph and get it back with updated values for later use?	Yes. You have complete control over what is exchanged in both directions, and the recommendation is to keep unnecessary context out rather than sharing everything by default.
komal	So research is where you would use the tools?	Yes, that is exactly the node that becomes agentic once you give it search tools.
imtiaaz	Semantic routing and embeddings do not really fit into LangGraph, do they? The routing here is mostly hardcoded.	They fit fine. Wrap them as logic inside a node, or make them their own router node. Shanmukhh's answer in the chat went further and is worth reading in full.

Key Insight (from shanmukhh vadrevu)

On imtiaaz's question: a dedicated router node can convert user input into embeddings and identify intent by semantic similarity rather than rigid keyword rules, then the graph routes execution to the correct downstream nodes before making expensive LLM calls. That replaces hardcoded rules with dynamic, context aware decisions, and it saves money at the same time.

9. Is It a Workflow or an Agent? The Orchestration Question

The cohort spotted the gap between "LangGraph builds agents" and what was actually on screen, and pushing on it produced the clearest section of the session.

Asked by	Question	Answer or takeaway
Amit Patil	As we are modelling every step manually, can we call this a workflow and not an agent?	Dhaval agreed outright. It is a workflow. There is a little agentic behaviour in the needs_work loop, but deterministic nodes calling an LLM make a workflow application.
Mugdha	We never defined an agent anywhere. Where is it?	That is the magic. There is no agent object. A node with the right logic and tools becomes the agent, without ever being declared as one.
Goutham	Can we give a node an additional LLM instance where required, for example an embedding model?	Yes. A node is a function, so anything you can call from a function can live in a node.
Yasmin Iqbal	Can every node connect to multiple tools?	Yes.
Goutham	Could you give a summary of what we did, explained to a ten year old?	We invoked the graph with a question. It started at the start node and followed the arrows, node by node, until it reached the end. It is just like llm.invoke, only with a route.
Ankit Gupta	Do we have utilities to reduce writing lengthy code, like in LangChain?	Yes.

10. Standout Participant Insights

Several people in the chat produced explanations good enough to keep. These are theirs.

From	The insight
Saquib Siddiqui	The whole first agent in one sentence: we generate something, score it, and if it meets our threshold we approve it. If it does not, we retry for a set period. If it eventually succeeds we approve, otherwise we give up. Then encapsulate all of that in a graph. It looks like a DAG.
Saquib Siddiqui	LangGraph looks like Keras, where instead of adding layers we are adding nodes.

From	The insight
Karthik Prabhu	On whether the graph is like a linked list: it is more precisely a directed, cyclical, attributed graph. Also flagged the resemblance to Redux, and to Temporal and Camunda on the orchestration side.
Amit Patil	Posted a full summary of the build sequence mid session, state to nodes to add_node to add_edge to conditional edges, and asked for it to be checked. Srikanth's reply: perfect, you got it. Worth reading as a self test.
Sandip	On why we are not using create_agent: now you are going inside the mind of the agent.
Sandip	On the first agent's flow: node one generates a draft and adds it to state, node two scores it and adds the score, node three revises according to the score, and this repeats until the goal is achieved at good quality.
shanmukhh vadrevu	Workflow automation handles specific tasks inside a defined process. Orchestration manages and governs the entire end to end system.
Yasmin Iqbal	With LangGraph the flow cannot be missed or mixed up, which reduces hallucinations and wasted iterations.
Priyaranjan Singh / ajit behera	Independently landed on the two best comparisons for what the graph resembles: Airflow, and step functions. Both were accepted.
Karthik Prabhu	On sustaining focus through long sessions: put your phone away, and avoid multitasking.

11. Setup Notes Worth Keeping

Situation	What to do
pip dependency resolver error in Colab	Reported by Tony, Pujan, Renganathan, saikrishna and Kaviarasu, and seen on the presenter's screen too. It is a version conflict warning between google colab and requests. It does not affect the notebook. Carry on.
Notebook appears stuck after entering the Groq key	Press Enter. Answered by Vivek and CK Grandhi within seconds of the question, before any mentor saw it.
Changing the score threshold does not trigger more iterations	Murthy raised this after setting the threshold to 8 and then higher. Syed's fix: re run the notebook from <code>builder = StateGraph()</code> onwards, since the old compiled graph is still in memory.
Mermaid graph render fails	Visualising the graph makes an API call. Without network access you will get an error. The graph itself is fine.

Situation	What to do
langchain community deprecation	Flagged by Murthy. Expect notebook dependencies to need refreshing over time.

Worth Noting

A large share of today's answers came from other participants, not the mentors: Syed, Karthik Prabhu, Amit Patil, Mugdha, chaithanya krishna, Sandip, shanmukhh, Vivek and CK Grandhi all answered questions correctly and fast. The team thanked several of them by name in the chat. If you were stuck today, the chat was very likely already ahead of you.